

VERIQTA

Infrastructure
as Code.
Reliability as
Standard.

TERRAFORM

PRODUCTION HANDBOOK

VOLUME 2

ADVANCED TERRAFORM IN PRODUCTION

★
ADVANCED.
ENTERPRISE.
PRODUCTION
READY.
★ ★



Advanced Patterns



Enterprise Best Practices



Team Workflows



Security & Compliance



Multi-Account & Multi-Region



CI/CD & Automation



Scalability & Performance



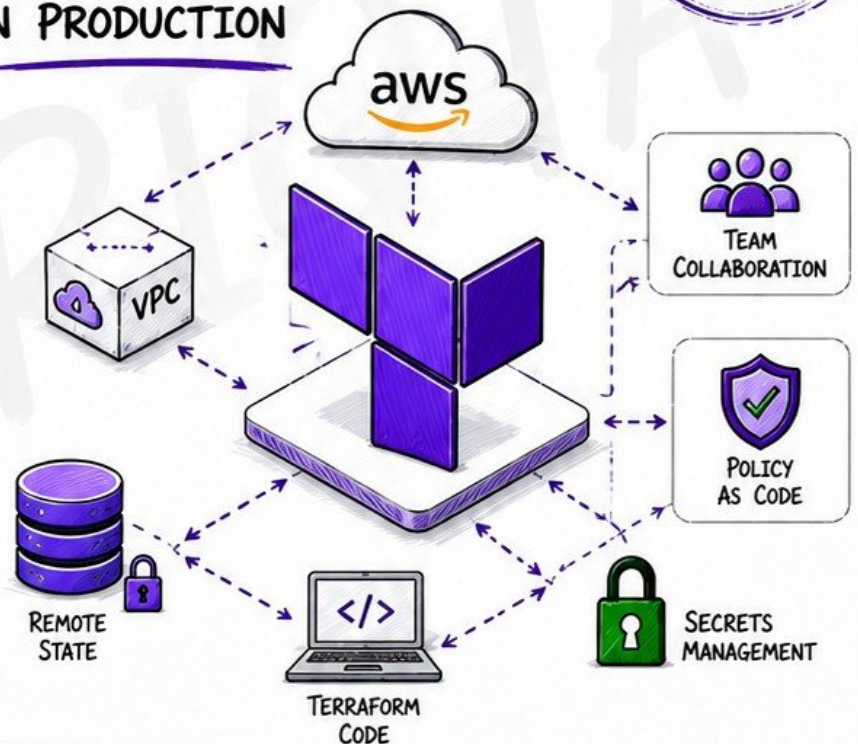
Real-World Case Studies



Capstone Project



Interview Preparation



RELIABLE

Build infrastructure
that you can trust
in production.



SCALABLE

Design for growth.
Deploy with
confidence.



COLLABORATIVE

Work together.
Version control.
Share knowledge.



AUTOMATED

Eliminate manual
work. Reduce risk.
Increase speed.



CLOUD AGNOSTIC

Works across AWS,
Azure, GCP and
more.

★ CODE IT. PLAN IT. APPLY IT. OWN IT. ★

MADE FOR
ENGINEERS.
BY ENGINEERS.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA

1. ADVANCED TERRAFORM OVERVIEW

LEVEL UP!

Move beyond basics and start thinking like a Senior Terraform Engineer!



★ WHAT IS ADVANCED TERRAFORM?

Advanced Terraform is about designing, operating and scaling Infrastructure as Code in production at enterprise scale with reliability, security and automation.

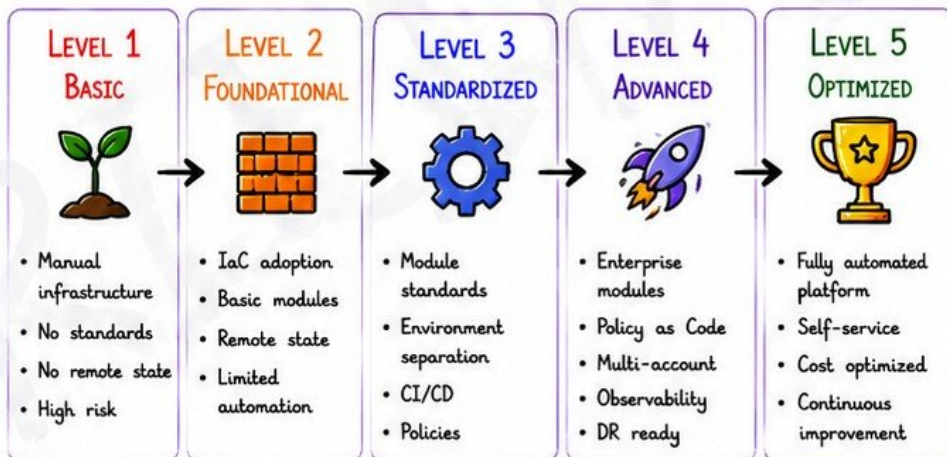


ENTERPRISE IAC PRINCIPLES

- ✓ Code is the source of truth
- ✓ Consistent, repeatable & versioned
- ✓ Secure by design
- ✓ Automated everything
- ✓ Collaborative and transparent
- ✓ Scalable and modular
- ✓ Continuously validated
- ✓ Governed and compliant
- ✓ Resilient and observable
- ✓ Cost efficient



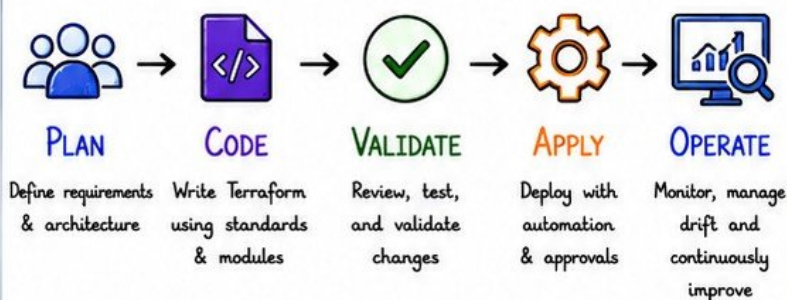
PRODUCTION MATURITY MODEL



GOAL:

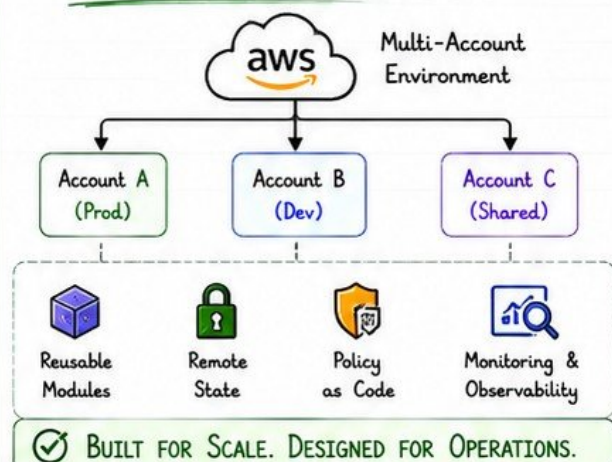
Build reliable, secure, and scalable infrastructure that enables your business to move fast and operate with confidence.

TEAM WORKFLOWS



COLLABORATION + AUTOMATION = RELIABILITY

LARGE-SCALE INFRASTRUCTURE



FOCUS: Build once. Use everywhere. Operate forever.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

VERIQTA

2. TERRAFORM WORKSPACES

REMEMBER!

Workspaces help you manage multiple environments with one configuration.



★ WHAT ARE TERRAFORM WORKSPACES?

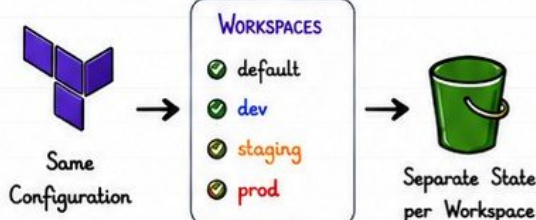
Workspaces allow you to manage multiple environments using the same configuration and state file.

MULTIPLE ENVIRONMENTS EXAMPLE

WORKSPACE	PURPOSE	EXAMPLE
default	Local / Testing	terraform workspace new default
dev	Development	terraform workspace new dev
staging	Staging / UAT	terraform workspace new staging
prod	Production	terraform workspace new prod

- ✓ Each workspace has its own state file and resource instances.

How It Works



One codebase.
Multiple environments.
Isolated state.
Clean operations.

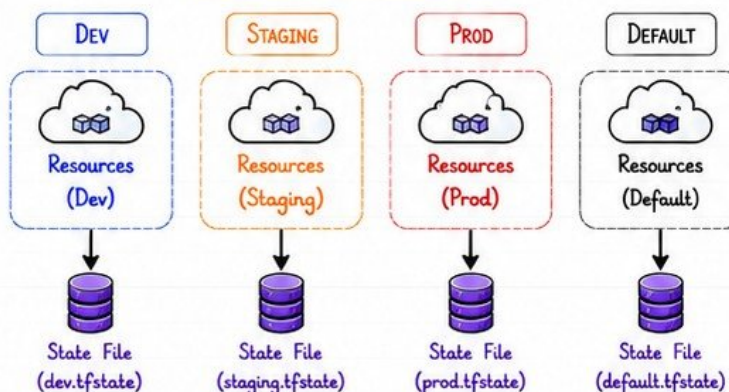
COMMON WORKSPACE COMMANDS

terraform workspace list # List all workspaces
terraform workspace new dev # Create a workspace
terraform workspace select dev # Switch workspace
terraform workspace show # Show current workspace
terraform workspace delete dev # Delete workspace
terraform workspace select default # Switch back to default



The default workspace is created automatically. You cannot delete it.

ENVIRONMENT ISOLATION



Isolation ensures changes in one environment NEVER affect another.

BEST PRACTICES

- ✓ Use workspaces for environments (dev, staging, prod).
- ✓ Keep state files separate per workspace.
- ✓ Use remote state backend (S3 + DynamoDB).
- ✓ Restrict access to prod workspace.
- ✓ Use variables for environment-specific values.
- ✓ Use naming conventions for resources.
- ✓ Automate workspace selection in CI/CD.

COMMON MISTAKES

- ✗ Using workspaces for multi-account separation.
- ✗ Hardcoding values per workspace.
- ✗ Manually switching workspace in pipelines.
- ✗ Mixing remote and local state.
- ✗ Deleting workspace without backup.
- ✗ Storing secrets in variables.
- ✗ No access control on prod workspace.

CLI EXAMPLES

```
# Create and switch to dev
terraform workspace new dev

# Switch to prod
terraform workspace select prod

# See current workspace
terraform workspace show

# List all workspaces
terraform workspace list
```

PRO TIP

Always run terraform workspace show before terraform apply!

WORKSPACE vs BACKEND

→ Backend = Where state is stored
→ Workspace = How state is separated
→ Together = Scalable & safe IaC

VERIQTA

3. DYNAMIC BLOCKS

REMEMBER!

Dynamic blocks help you generate repeatable nested blocks using variables and loops.

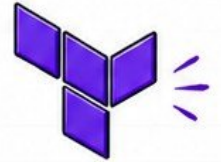


★ WHY DYNAMIC BLOCKS MATTER

- ✔ Infrastructure often contains repeatable nested blocks.
- ✔ Dynamic blocks let you generate them from variables.
- ✔ Reduces code duplication and improves maintainability.
- ✔ Makes modules flexible and production-ready.
- ✔ Essential for enterprise scale and multi-environment IaC.

WHEN TO USE DYNAMIC BLOCKS

- ✔ When you don't know the number of blocks.
- ✔ When the content comes from variables or maps.
- ✔ When building reusable, scalable modules.
- ✗ When the number of blocks is fixed and small.
- ✗ When a simple argument is enough.



SYNTAX

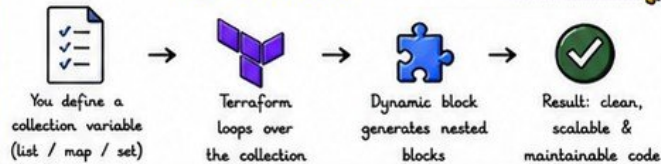
```
dynamic "BLOCK_LABEL" {  
  for_each = COLLECTION  
  iterator = ITEM # optional (default is the label)  
  content {  
    # arguments of the nested block  
    ARGUMENT1 = ITEM.value.ARG1  
    ARGUMENT2 = ITEM.value.ARG2  
  }  
}
```

KEY PARTS

- **for_each**: collection to iterate over (list, set, map)
- **iterator**: name used inside content (optional)
- **content**: defines the nested block arguments



HOW IT WORKS



NESTED BLOCKS EXAMPLE

```
resource "aws_security_group" "example" {  
  name = "example-sg"  
  description = "SG with dynamic ingress rules"  
  vpc_id = var.vpc_id  
  
  dynamic "ingress" {  
    for_each = var.ingress_rules  
    content {  
      from_port = ingress.value.from_port  
      to_port = ingress.value.to_port  
      protocol = ingress.value.protocol  
      cidr_blocks = ingress.value.cidr_blocks  
      description = ingress.value.description  
    }  
  }  
}
```

Creates one ingress block for each item in var.ingress_rules



PRODUCTION EXAMPLES

① MULTIPLE INGRESS RULES

Variable

```
variable "ingress_rules" {  
  type = list(object({  
    from_port = number  
    to_port = number  
    protocol = string  
    cidr_blocks = list(string)  
    description = string  
  }))  
}
```

Use Case

Create SG ingress rules from a list.



② MULTIPLE TAGS

Variable

```
variable "tags" {  
  type = map(string)  
}
```

Resource Example

```
resource "aws_instance" "example" {  
  ami = var.ami  
  instance_type = var.instance_type  
  
  dynamic "tag" {  
    for_each = var.tags  
    content {  
      key = tag.key  
      value = tag.value  
    }  
  }  
}
```

Creates a tag block for each key/value pair.



③ MULTIPLE SUBNETS IN A MODULE

Variable

```
variable "subnets" {  
  type = list(string)  
}
```

Resource Example

```
resource "aws_route_table_association" "example" {  
  dynamic "subnet_id" {  
    for_each = var.subnets  
    content {  
      subnet_id = subnet_id.value  
    }  
  }  
  route_table_id = var.route_table_id  
}
```

Associates the route table with all subnets in the list.



★ BEST PRACTICES

- ✔ Use descriptive iterator names.
- ✔ Validate input variables with proper types.
- ✔ Keep dynamic blocks simple and focused.
- ✔ Document variable structure clearly.
- ✔ Test with terraform plan before apply.



COMMON MISTAKES

- ✗ Forgetting for_each or using wrong collection.
- ✗ Using dynamic block when static is enough.
- ✗ Wrong iterator reference inside content {}.
- ✗ Complex logic inside dynamic blocks.
- ✗ Not validating variable structure.



PRO TIP

Dynamic blocks + for_each are your best friends for building flexible, reusable, and enterprise-grade Terraform modules.

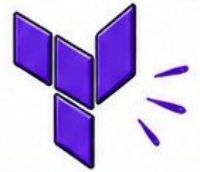


VERIQTA

4. LIFECYCLE META-ARGUMENTS

REMEMBER!

- ✓ Control resource behavior
- ✓ Prevent downtime
- ✓ Protect critical resources
- ✓ Reduce blast radius



★ WHAT ARE LIFECYCLE META-ARGUMENTS?

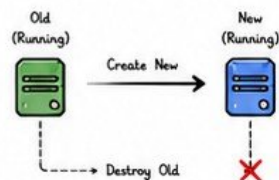
Lifecycle meta-arguments control how Terraform creates, updates and destroys resources. They are essential for zero-downtime, safety and operational stability in production.

1. CREATE_BEFORE_DESTROY

Creates the new resource before destroying the old one. Minimizes downtime during replacements.

```
resource "aws_instance" "web" {  
  ami           = var.ami  
  instance_type = var.instance_type  
  
  lifecycle {  
    create_before_destroy = true  
  }  
}
```

How it Works



Use Cases

- EC2 instances
- RDS instances
- Load balancers
- Auto Scaling Groups



Ensures high availability and zero downtime.

2. PREVENT_DESTROY

Prevents Terraform from destroying the resource. Protects critical infrastructure from accidental deletion.

```
resource "aws_s3_bucket" "prod_data" {  
  bucket = "prod-important-data"  
  
  lifecycle {  
    prevent_destroy = true  
  }  
}
```

Behavior



Terraform apply will **FAIL** if the resource needs to be destroyed.

Use Cases

- S3 buckets with important data
- Production databases
- Critical networking components



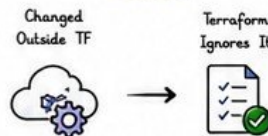
Adds a safety brake for production.

3. IGNORE_CHANGES

Ignore specific attribute changes made outside Terraform. Prevents unnecessary updates and drift noise.

```
resource "aws_instance" "app" {  
  ami           = var.ami  
  instance_type = var.instance_type  
  tags          = var.tags  
  
  lifecycle {  
    ignore_changes = [  
      tags["LastBackup"],  
      user_data  
    ]  
  }  
}
```

Behavior



Use Cases

- Tags managed by other systems
- Auto-generated values
- Fields updated by external tools



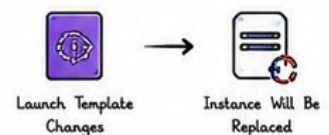
Reduces unnecessary diffs and prevents churn.

4. REPLACE_TRIGGERED_BY

Forces replacement of this resource when another resource changes.

```
resource "aws_instance" "app" {  
  ami           = var.ami  
  instance_type = var.instance_type  
  
  lifecycle {  
    replace_triggered_by = [  
      aws_launch_template.app  
    ]  
  }  
}
```

Behavior



Use Cases

- AMI updates
- Launch template changes
- Security baseline updates



Keeps resources in sync with dependent changes.

PRODUCTION SCENARIOS

SCENARIO	RECOMMENDED META-ARGUMENT	WHY	EXAMPLE
Update load balancer	create_before_destroy	Avoid downtime during replacement	ALB replacement without traffic loss
Protect production database	prevent_destroy	Prevent accidental deletion	RDS, Aurora, DynamoDB tables
Tags managed by external tool	ignore_changes	Avoid unnecessary Terraform updates	Cost allocation or backup tags
Roll instances when AMI changes	replace_triggered_by	Ensure instances use latest AMI	Trigger replacement when AMI updates

PRO TIP

Use lifecycle rules intentionally. Too much control can hide real issues.

BEST PRACTICES

- ✓ Document why you use each rule.
- ✓ Review regularly in code reviews.
- ✓ Test lifecycle behavior in non-prod first.



COMMON MISTAKES

- ✗ Overusing ignore_changes
- ✗ Forgetting prevent_destroy on critical resources
- ✗ Not understanding replacement impact



KEY TAKEAWAY

Lifecycle meta-arguments give you control, safety and stability in real-world production.



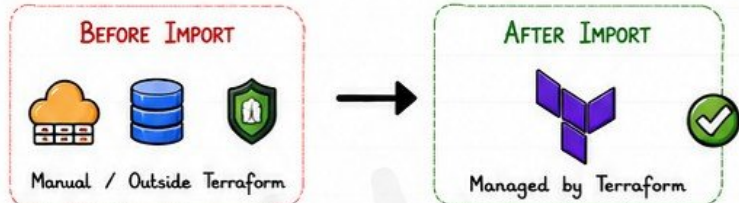
5. IMPORT EXISTING INFRASTRUCTURE

REMEMBER!

- ✓ Import brings existing resources under Terraform management.
- ✓ No infrastructure is recreated during import.

★ WHY IMPORT?

You can bring existing, manually created infrastructure under Terraform management without recreating or destroying it. This allows you to adopt Terraform incrementally and safely.



1 TERRAFORM IMPORT (CLI)

Classic way to import a single resource.

```
# Syntax
terraform import <address> <resource-id>

# Example: Import an existing S3 bucket
terraform import aws_s3_bucket.logs my-existing-bucket
```

Steps:

- 1 Define the resource block in your configuration.
- 2 Run `terraform import <address> <id>`.
- 3 Run `terraform plan` to verify.
- 4 Apply only if changes are required.



2 IMPORT BLOCKS (TERRAFORM 1.5+)

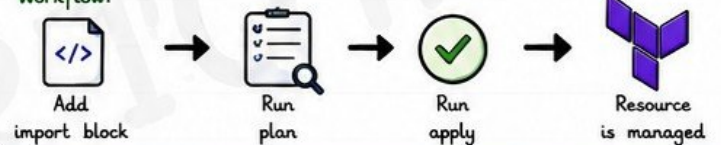
Declarative import - define in code, then run plan/apply.

```
resource "aws_s3_bucket" "logs" {
  bucket = "my-existing-bucket"
}

import {
  to = aws_s3_bucket.logs
  id = "my-existing-bucket"
}
```

- ✓ Stored in configuration
- ✓ Version controlled
- ✓ Repeatable & auditable

Workflow:



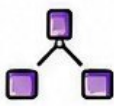
3. MIGRATING MANUAL INFRASTRUCTURE (RECOMMENDED APPROACH)

1. Inventory



List all existing resources and dependencies.

2. Map



Map resources to Terraform providers and resource types.

3. Author



Write resource blocks that match existing settings.

4. Import



Import resources using CLI or import blocks.

5. Validate



Run plan and ensure no changes (0 to add, 0 to change, 0 to destroy).

6. Refactor



Move into modules, add tagging, outputs and standards.

7. Govern



Apply policies, enforce guardrails and monitoring.

4. COMMON CHALLENGES



- | | |
|---------------------------|--|
| ✗ Resource ID not known | Finding the correct ID format can be difficult. |
| ✗ State mismatch | Configuration doesn't match the actual resource. |
| ✗ Drift after import | Existing settings cause unexpected diffs. |
| ✗ Missing dependencies | Related resources not imported cause errors. |
| ✗ Permissions | Insufficient IAM permissions to read the resource. |
| ✗ Provider/version issues | Older providers may not support the resource. |
| ✗ Accidental recreation | Improper config may try to replace the resource. |

5. BEST PRACTICES

- ✓ Always backup your state before importing.
- ✓ Import one resource at a time.
- ✓ After import, run plan and verify zero changes.
- ✓ Use import blocks for repeatability and auditability.
- ✓ Document the import process and resource IDs.
- ✓ Align tags and naming conventions after import.
- ✓ Refactor into modules after validating.
- ✓ Use remote state backend for team consistency.

CHECKLIST

- ✓ Inventory
- ✓ Map
- ✓ Author
- ✓ Import
- ✓ Validate
- ✓ Refactor
- ✓ Govern



IMPORT DOES NOT CHANGE INFRASTRUCTURE. IT ONLY BRINGS IT UNDER TERRAFORM MANAGEMENT.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA

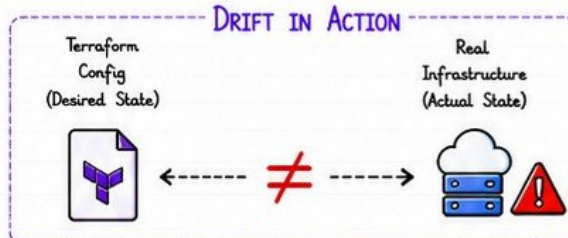
6. MANAGING DRIFT

REMEMBER!

- ✓ Drift is inevitable.
- ✓ Detection is critical.
- ✓ Automation is the key.

★ WHAT IS DRIFT?

Drift occurs when the real infrastructure (Actual state) differs from what is defined in your Terraform configuration (desired state). This happens due to manual changes, external tools, or provider updates.



🔍 1. DRIFT DETECTION

- ✓ Terraform compares the state file with real infrastructure.
- ✓ Detects changes made outside of Terraform.
- ✓ Identifies missing, modified, or extra resources.
- ✓ First line of defense for production stability.

COMMON CAUSES OF DRIFT



> 2. TERRAFORM PLAN

terraform plan is the primary tool for detecting drift.

Example Output (Drift Detected)

```
~ aws_instance.web
  instance_type: t3.micro => t3.small
  tags["Owner"]: "DevOps" => "SRE"
+ aws_security_group_extra
  (resource created outside Terraform)
- aws_s3_bucket.old_bucket
  (resource deleted outside Terraform)
```

Plan output clearly shows what has changed.
Always review before applying.

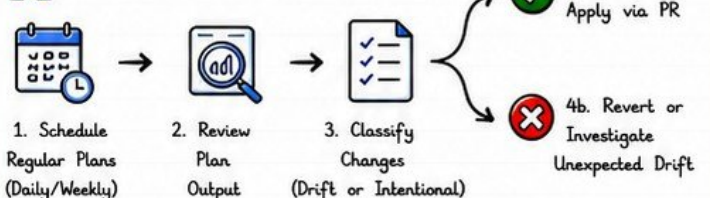
🔄 3. REFRESH BEHAVIOR

- ✓ Terraform automatically refreshes state before plan/apply.
- ✓ It queries the real infrastructure from the provider.
- ✓ Updates the state file with the latest remote data.
- ✓ Does NOT modify your infrastructure.

```
$ terraform plan -refresh=true      # default
$ terraform plan -refresh=false     # skip refresh (not recommended)
$ terraform apply -refresh-only     # update state without changes
```

Refresh ensures your state file reflects reality.
Never skip refresh in production.

🔄 4. OPERATIONAL WORKFLOWS



Goal: Keep Infrastructure Consistent, Predictable & Auditable

5. PRODUCTION EXAMPLES

EXAMPLE	DRIFT SCENARIO	PLAN OUTPUT (EXAMPLE)	ACTION
EC2 Instance	Instance type changed manually from t3.micro to t3.small	~ instance_type: t3.micro => t3.small	Review change, update config or revert manually.
Security Group	Inbound rule added manually allowing port 3389	+ aws_security_group_rule.manual_rule	Remove manual rule or import it.
S3 Bucket	Bucket deleted outside Terraform	- aws_s3_bucket.app_bucket	Recreate (if needed) or adjust config.
Resource Tag	Tag "Owner" changed manually from DevOps to SRE	~ tags["Owner"]: "DevOps" => "SRE"	Decide desired value and apply.
RDS Database	DB instance upgraded manually to a larger class	~ instance_class: db.t3.micro => db.t3.medium	Update config to match or revert change.

BEST PRACTICES

- ✓ Run terraform plan regularly (automate it).
- ✓ Never edit infrastructure outside Terraform.
- ✓ Use drift detection before every apply.
- ✓ Keep state file secure and accurate.
- ✓ Use monitoring/alerts for critical drifts.
- ✓ Document all intentional changes.

COMMON MISTAKES

- ✗ Making manual changes in production.
- ✗ Ignoring small drifts (they compound).
- ✗ Skipping plan before apply.
- ✗ Running plan with -refresh=false.
- ✗ Not updating state after external changes.

PRO TIP

- Automate drift detection with:
- ✓ Terraform Cloud/Enterprise
- ✓ CI/CD pipelines
- ✓ AWS Config / CloudTrail
- ✓ Slack / Email alerts

Detect early, fix fast, stay sane!

7. TERRAFORM CLOUD (TFC)

REMEMBER!

- ✓ Centralize your IaC workflows
- ✓ Automate safely at scale
- ✓ Enforce policy consistently
- ✓ Improve collaboration & visibility

★ WHAT IS TERRAFORM CLOUD?

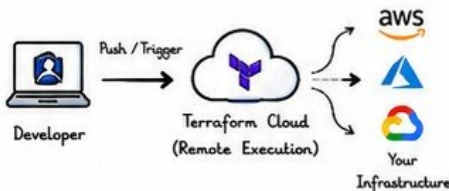
Terraform Cloud (TFC) is a SaaS platform by HashiCorp that provides remote execution, state management, collaboration, policy enforcement, and visibility for Terraform at scale.

- ☁ Remote Execution
- 🔒 State Management
- 👥 Team Collaboration
- 📜 Policy as Code
- 📊 Run & Cost Insights



1. REMOTE EXECUTION

- ✓ Plans and applies run on TFC infrastructure.
- ✓ No need for local Terraform CLI.
- ✓ Secure, consistent, and auditable.



✓ Credentials never leave TFC.
Everything runs in a secure, isolated environment.

2. WORKSPACES

- ✓ Workspaces map to environments.
- ✓ Isolate state, variables, and runs.
- ✓ Easy to manage multiple envs.

Workspace	Environment
dev	Development
staging	Staging
prod	Production

💡 Each workspace has its own state, variables, and settings.

3. VARIABLES

- ✓ Store variables in Terraform Cloud.
- ✓ Support for sensitive (encrypted) variables.
- ✓ Set by workspace, variable sets, or CLI.

VARIABLE SOURCES



🔒 Sensitive values are encrypted at rest and in transit.

4. POLICY ENFORCEMENT

- ✓ Use Sentinel to write & enforce policies.
- ✓ Prevent non-compliant changes.
- ✓ Apply governance guardrails consistently.

POLICY EXAMPLES

- ✓ Restrict instance types
- ✓ Require tags on all resources
- ✓ Disallow public S3 buckets
- ✓ Enforce approved providers/versions

🛡️ Policies run during plan and apply.
Non-compliant changes are blocked.

5. TEAM COLLABORATION

- ✓ Teams can collaborate safely and transparently.
- ✓ Role-based access control (RBAC).
- ✓ Run history, audit logs, and notifications.

TFC COLLABORATION FLOW



🎯 Full visibility. Full audit. Full control.

TERRAFORM CLOUD KEY COMPONENTS

Organizations

Top-level container for users, teams, workspaces, and policies.

Workspaces

Isolated environments with their own state, variables, and settings.

State Management

Remote state stored securely, with locking, encryption, and versioning.

Runs

Automated plan and apply workflows triggered by VCS or manually.

Policy Sets

Group of Sentinel policies applied across multiple workspaces.

Teams & RBAC

Granular access control for collaboration and security.

Notifications

Email, Slack, or webhook alerts for important events.

BEST PRACTICES

- ✓ Use remote state always.
- ✓ Map each environment to a workspace.
- ✓ Use variable sets for shared values.
- ✓ Enforce policies as code.
- ✓ Review every plan before apply.
- ✓ Use least-privilege access.



COMMON MISTAKES

- ✗ Using the default workspace for everything.
- ✗ Storing secrets in plain text variables.
- ✗ Skipping policy enforcement.
- ✗ Giving excessive permissions.
- ✗ Not reviewing plans before apply.



PRO TIP

Automate your workflow:

- ✓ GitHub / GitLab integration
- ✓ Auto plan on PR
- ✓ Policy checks
- ✓ Manual approval
- ✓ Auto apply after approval

Safe. Scalable. Repeatable.



VERIQTA

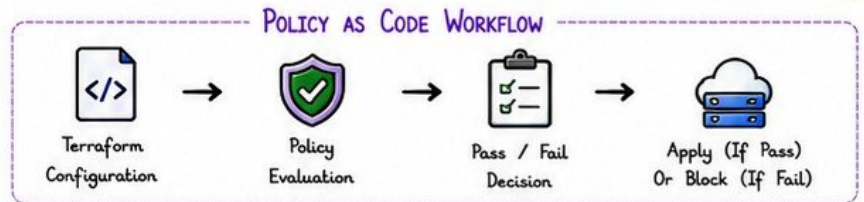
8. POLICY AS CODE

REMEMBER!

- ✓ Policies prevent bad infrastructure.
- ✓ Automate compliance & governance.
- ✓ Fail early, not in production.
- ✓ Consistent rules at enterprise scale.

★ WHAT IS POLICY AS CODE?

Policy as Code (PaC) lets you define rules that your infrastructure must follow. Terraform checks these policies before changes are applied, ensuring security, compliance, and governance automatically.



1. SENTINEL (TERRAFORM CLOUD)

- ✓ Native to Terraform Cloud.
- ✓ Built by HashiCorp.
- ✓ Enforces policies during plan.
- ✓ Deep integration with TFC.

SENTINEL EXAMPLE

```
import "tfplan/v2" as tfplan

main = rule {
  all tfplan.resource_changes as rc {
    rc.type is not "aws_s3_bucket" or
    (rc.change.after.versioning[0].enabled is true)
  }
}
```

This policy requires all S3 buckets to have versioning enabled.

2. OPEN POLICY AGENT (OPA)

- ✓ Open source policy engine.
- ✓ Cloud agnostic (works everywhere).
- ✓ Uses Rego language.
- ✓ Can be integrated with Terraform.

OPA / REGO EXAMPLE

```
package terraform.aws

deny[msg] {
  input.resource.type == "aws_s3_bucket"
  not input.resource.values.versioning[0].enabled
  msg := "S3 bucket must have versioning enabled"
}
```

This policy denies any S3 bucket without versioning enabled.

3. POLICY COMPARISON

Feature	Sentinel (TFC)	OPA (Open Source)
Provider	HashiCorp	Open Source Community
Language	Sentinel	Rego
Integration	Native in TFC	Any CI/CD, Terraform, K8s
Use Case	Terraform Cloud Governance	Multi-cloud Enterprise Policies
Learning Curve	Easy	Medium
Deployment	TFC Only	Anywhere

Choose based on your environment and governance needs.

4. COMPLIANCE

- ✓ Enforce regulatory standards (PCI, HIPAA, ISO).
- ✓ Ensure encryption, tagging, logging, and backup.
- ✓ Prevent non-compliant resources.
- ✓ Generate audit-ready evidence.

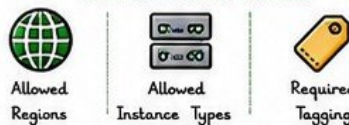
COMPLIANCE CHECKS EXAMPLES



5. GOVERNANCE

- ✓ Standardize infrastructure across teams.
- ✓ Enforce naming conventions and tagging.
- ✓ Control allowed regions, instance types, and services.
- ✓ Reduce risk and configuration drift.

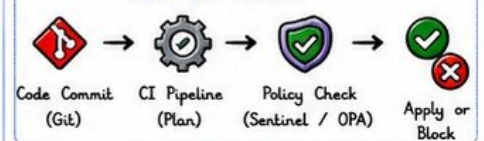
GOVERNANCE RULES EXAMPLES



6. ENTERPRISE WORKFLOWS

- ✓ Policies evaluated automatically in CI/CD.
- ✓ Fail fast before changes reach production.
- ✓ Integrate with Git, Terraform Cloud, and OPA.
- ✓ Central policy library and version control.

ENTERPRISE WORKFLOW



7. COMMON POLICY CATEGORIES



Security
Encryption, IAM, Security Groups, Secrets, etc.



Cost
Instance types, Resource limits, Tagging, Budgets.



Reliability
Multi-AZ, Backup, Auto Scaling, Monitoring.



Compliance
Regulations, Standards, Audit controls.



Operations
Logging, Alerts, Maintenance Windows.



Networking
VPC, Subnets, Private Access, Firewall rules.



Data Protection
Encryption, Backup Retention, Data Classifications.

BEST PRACTICES

- ✓ Start small, enforce critical policies first.
- ✓ Version control your policies.
- ✓ Test policies with unit tests.
- ✓ Document policies and exceptions.
- ✓ Review and update policies regularly.



COMMON MISTAKES

- ✗ Overly restrictive policies (block everything).
- ✗ No exceptions process.
- ✗ Not testing policies before enforcing.
- ✗ Scattered policies across teams.
- ✗ Ignoring drift and policy updates.



PRO TIP

Shift left! Evaluate policies in the plan phase to catch issues early and save time & cost.

- ✓ Use Policy Libraries
- ✓ Automate Everything
- ✓ Monitor Policy Violations
- ✓ Keep Policies Simple & Clear



VERIQTA

9. TERRAFORM TESTING

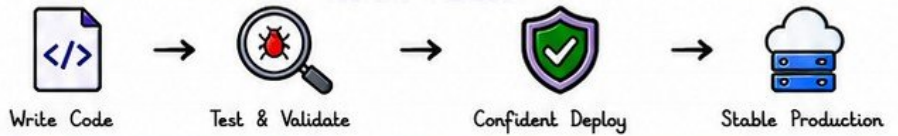
REMEMBER!

- ✓ Test early, test often.
- ✓ Automate validation.
- ✓ Prevent bad configs from reaching production.

★ WHY TEST TERRAFORM?

Testing helps you catch errors early, enforce standards, validate behavior, and prevent drift or outages in production.

TESTING MINDSET



1. VALIDATE

Validate Terraform configuration syntax and internal consistency.

COMMAND

```
$ terraform validate
```

CHECKS

- ✓ Syntax errors
- ✓ Missing required arguments
- ✓ Invalid resource references
- ✓ Type mismatches



Run validate on every commit. Fast, lightweight, and catches basic issues.

2. TEST (PLAN TESTING)

Test the execution plan to ensure expected changes before applying.

COMMAND

```
$ terraform plan -out=tfplan  
$ terraform show -json tfplan > plan.json
```

WHAT TO CHECK

- ✓ No unexpected resources
- ✓ Correct resource counts
- ✓ No destructive actions (delete/replace)
- ✓ Tagging, naming, and values



Automate checks using OPA, Sentinel, or custom scripts on the plan output.

3. TERRATEST (GO)

Terratest is an open-source Go framework for testing Terraform modules.

EXAMPLE (GO TEST)

```
$ go test ./test/...  
-v -timeout 30m
```

WHAT IT CAN TEST

- ✓ Resources created as expected
- ✓ Outputs have correct values
- ✓ Security groups / policies
- ✓ Tags, encryption, networking



Terratest = Unit + Integration testing for your infrastructure code.

4. INTEGRATION TESTING

Verify your infrastructure actually works end-to-end.

EXAMPLE TESTS

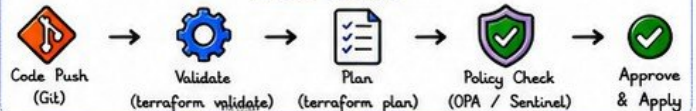


Don't just create it. Prove it works like real users and applications will.

5. CI/CD VALIDATION

Automate testing in your CI/CD pipeline to block bad changes.

TYPICAL PIPELINE



FAIL THE PIPELINE IF:

- Validation fails
- Plan has destructive changes
- Policy violations
- Tests fail (Terratest / Integration)



BENEFITS

- ✓ Prevent bad deployments
- ✓ Ensure compliance
- ✓ Improve confidence
- ✓ Reduce outages

6. TESTING PYRAMID FOR INFRASTRUCTURE



Unit Tests (Module Logic)

Test variable validation, locals, outputs. Example: `terraform validate`, `tflint`



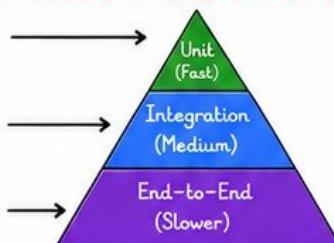
Integration Tests (Components)

Test resources and modules in isolation. Example: Terratest modules



End-to-End Tests (Full Stack)

Test the complete infrastructure workflow. Example: Real-world scenarios



BEST PRACTICES

- ✓ Test small, fail fast.
- ✓ Automate everything.
- ✓ Use realistic test data.
- ✓ Keep tests independent.
- ✓ Run in ephemeral environments.



COMMON MISTAKES

- ✗ Skipping tests
- ✗ Testing only happy path
- ✗ Only manual plan review
- ✗ Ignoring policy checks



PRO TIP

Good tests don't just check if it deploys, they prove it behaves, stays secure, and meets business requirements.



Test like your production depends on it. Because it does.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

10. CI/CD WITH TERRAFORM

AUTOMATE. VALIDATE. DEPLOY. REPEAT.

REMEMBER!

- ✓ Automate everything.
- ✓ Validate before you deploy.
- ✓ Use remote state & locking.
- ✓ Small changes. Safe deploys.

★ WHY CI/CD FOR TERRAFORM?

- ✓ Consistent, repeatable deployments.
- ✓ Catch issues early with automated checks.
- ✓ Collaborate safely with version control.
- ✓ Faster delivery with guardrails.
- ✓ Full audit trail of every change.

TERRAFORM CI/CD PIPELINE OVERVIEW



1. GITHUB ACTIONS

WORKFLOW OVERVIEW



EXAMPLE (PLAN JOB)

```
- name: Terraform Plan
run: |
  terraform init
  terraform validate
  terraform fmt -check
  terraform plan -out=tfplan
- name: Upload Plan
uses: actions/upload-artifact@v4
with:
  name: tfplan
  path: tfplan
```



GitHub
Actions



2. JENKINS

PIPELINE STAGES



EXAMPLE (JENKINSFILE)

```
stage('Plan') {
  steps {
    sh 'terraform init'
    sh 'terraform validate'
    sh 'terraform plan -out=tfplan'
    archiveArtifacts artifacts: 'tfplan'
  }
}
```



Jenkins



3. GITLAB CI

PIPELINE OVERVIEW



EXAMPLE (.GITLAB-CI.YML)

```
plan:
  script:
    - terraform init
    - terraform validate
    - terraform plan -out=tfplan
  artifacts:
    paths:
      - tfplan
```



GitLab CI



4. AZURE DEVOPS

PIPELINE OVERVIEW



EXAMPLE (YAML SNIPPET)

```
- task: TerraformCLI@1
  displayName: 'Terraform Plan'
  inputs:
    command: 'plan'
    environmentServiceName: 'Azure'
    publishPlanResults: 'true'
```



Azure DevOps

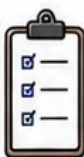
PRODUCTION GRADE TERRAFORM PIPELINE (EXAMPLE)



Use remote state, locking, secrets management and least-privilege access in every environment.

BEST PRACTICES

- ✓ Keep infrastructure code in Git.
- ✓ Run validate, fmt, tfLint on every commit.
- ✓ Always review plan before apply.
- ✓ Use remote state with locking.
- ✓ Enforce policies (Sentinel / OPA).
- ✓ Run automated tests (Terratest).
- ✓ Use protected environments for production.



COMMON MISTAKES

- ✗ Applying without reviewing the plan.
- ✗ Skipping validation or linting.
- ✗ Storing state locally.
- ✗ Hardcoding credentials.
- ✗ Using overly broad permissions.
- ✗ Not testing infrastructure changes.



PRO TIP

Treat your infrastructure like application code. Use the same DevOps discipline:

- ✓ Code Review
- ✓ Automated Testing
- ✓ Policy Enforcement
- ✓ Continuous Delivery
- ✓ Observability & Feedback



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

11. SECRETS MANAGEMENT

REMEMBER!

- ✓ Never hardcode secrets
- ✓ Encrypt everything
- ✓ Use least-privilege access.
- ✓ Rotate secrets regularly

★ WHY SECRETS MANAGEMENT?

Secrets include passwords, API keys, tokens, certificates, and private keys. Poor handling leads to breaches, outages, and compliance failures.

SECRETS MANAGEMENT PRINCIPLES



Secure
Storage



Controlled
Access



Rotation &
Expiration



Audit &
Monitoring



Least
Privilege

1. AWS SECRETS MANAGER



- ✓ Managed service to store, retrieve, and rotate secrets.
- ✓ Automatic rotation for databases & services.
- ✓ Integrated with IAM, CloudTrail, and CloudWatch.

EXAMPLE (TERRAFORM)

```
resource "aws_secretsmanager_secret" "db" {
  name = "prod/db/password"
}

resource "aws_secretsmanager_secret_version"
"db_version" {
  secret_id = aws_secretsmanager_secret.db.id
  secret_string = "superSecret123!"
}
```

USE CASES



2. HASHICORP VAULT



- ✓ Open-source secrets management.
- ✓ Dynamic secrets for databases, clouds, and apps.
- ✓ Fine-grained access with policies.

EXAMPLE (TERRAFORM)

```
provider "vault" {
  address = "https://vault.example.com"
  token = var.vault_token
}

resource "vault_kv_secret_v2" "app" {
  mount = "secret"
  name = "app/config"
  data_json = jsonencode({
    username = "appouser"
    password = "Str0ngP@ss!"
  })
}
```

USE CASES



3. AZURE KEY VAULT



- ✓ Cloud-native secrets store.
- ✓ Integrates with Azure AD, RBAC, and services.
- ✓ Supports keys, secrets, and certificates.

EXAMPLE (TERRAFORM)

```
resource "azurerm_key_vault" "kv" {
  name = "prod-kv"
  location = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  sku_name = "standard"
}

resource "azurerm_key_vault_secret" "db" {
  name = "db-password"
  value = "P@ssw0rd123"
  key_vault_id = azurerm_key_vault.kv.id
}
```

USE CASES



4. GCP SECRET MANAGER



- ✓ Store, manage, and access secrets securely.
- ✓ Automatic replication & strong IAM integration.
- ✓ Audit with Cloud Logging.

EXAMPLE (TERRAFORM)

```
resource "google_secret_manager_secret" "db" {
  secret_id = "prod-db-password"
  replication {
    automatic = true
  }
}

resource "google_secret_manager_secret_version"
"db_version" {
  secret = google_secret_manager_secret.db.id
  secret_data = "S3cureP@ss123"
}
```

USE CASES



5. SECRETS MANAGEMENT BEST PRACTICES



Never hardcode secrets in code, configs, or images.



Use IAM/RBAC and least-privilege access always.



Rotate secrets regularly and expire old ones.



Encrypt secrets at rest and in transit.



Enable audit logs and monitor access.



Store secrets in dedicated, managed services.



Use dynamic secrets whenever your platform supports it.



Review & test secret access in non-prod before production.

COMMON MISTAKES

- ✗ Storing secrets in Git repositories.
- ✗ Hardcoding secrets in Terraform or apps.
- ✗ Using overly broad IAM permissions.
- ✗ Not rotating or expiring secrets.
- ✗ Sharing secrets between environments.



SECURE SECRETS WORKFLOW



Automate the workflow and enforce via policy as code. Secure by default. Audit always.

PRO TIP

Combine secrets management with:

- ✓ Policy as Code (Sentinel / OPA)
- ✓ GitOps for secure delivery
- ✓ CI/CD secret scanning
- ✓ Regular rotation automation
- ✓ Monitoring and alerting

Security is a process, not a product.



VERIQTA

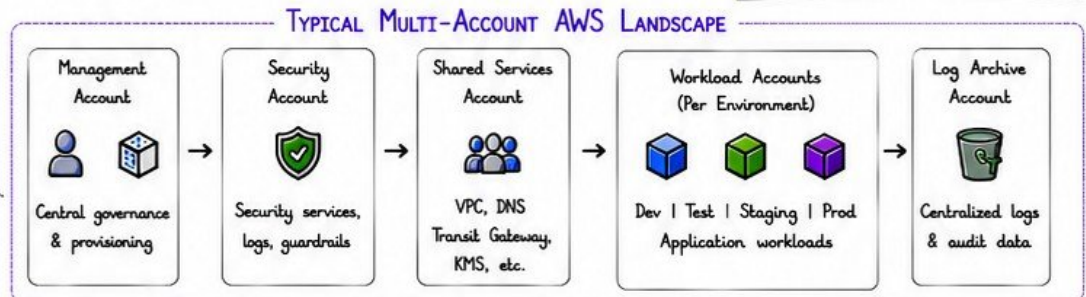
12. MULTI-ACCOUNT AWS DEPLOYMENTS

REMEMBER!

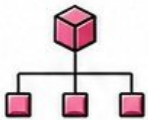
- ✓ Centralize & standardize.
- ✓ Use least-privilege access.
- ✓ Automate confidently.
- ✓ Monitor across all accounts.

★ WHY MULTI-ACCOUNT?

- ✓ Stronger security isolation.
- ✓ Separate billing and cost visibility.
- ✓ Environment separation (prod, dev, test).
- ✓ Compliance, governance & audit boundaries.
- ✓ Scalability for large enterprises.

**1. AWS ORGANIZATIONS**

- ✓ Group and manage AWS accounts.
- ✓ Apply Service Control Policies (SCPs).
- ✓ Centralized billing & reporting.
- ✓ Automate account creation.

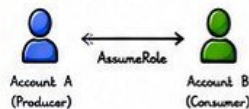
**EXAMPLE (ORG STRUCTURE)**

```

Root (OU)
├── Management (111111111111)
├── Security (222222222222)
├── Shared-Services (333333333333)
├── Workloads (444444444444)
│   ├── Dev (555555555555)
│   ├── Test (666666666666)
│   ├── Staging (777777777777)
│   └── Prod (888888888888)
  
```

2. CROSS-ACCOUNT ROLES

- ✓ Use IAM roles for secure access between accounts.
- ✓ Producer account trusts consumer account.
- ✓ AssumeRole with external ID for extra security.

**EXAMPLE (TRUST POLICY)**

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::222222222222:root"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "veriqta-external-id"
        }
      }
    }
  ]
}
  
```

3. PROVIDER ALIASES

- ✓ Use multiple provider blocks.
- ✓ Assign an alias per account.
- ✓ Deploy resources to target accounts easily.
- ✓ Keep code DRY and reusable.

EXAMPLE (PROVIDERS)

```

provider "aws" {
  alias  = "management"
  region = "us-east-1"
}

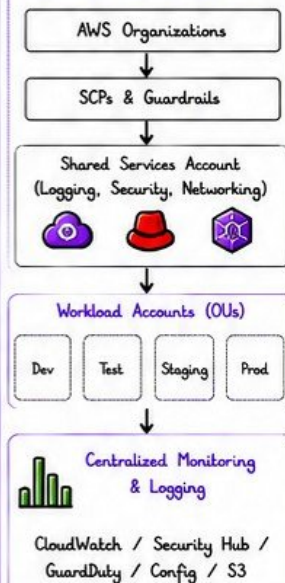
provider "aws" {
  alias  = "dev"
  region = "us-east-1"
  assume_role {
    role_arn = "arn:aws:iam::555555555555:role/dev-role"
    external_id = "veriqta-external-id"
  }
}

provider "aws" {
  alias  = "prod"
  region = "us-east-1"
  assume_role {
    role_arn = "arn:aws:iam::888888888888:role/prod-role"
    external_id = "veriqta-external-id"
  }
}

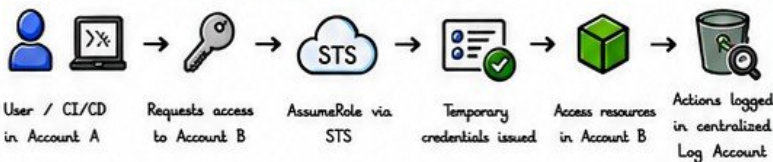
USAGE
module "vpc_dev" {
  source = "../modules/vpc"
  providers = { aws = aws.dev }
}
  
```

4. ENTERPRISE ARCHITECTURE

- ✓ Standard landing zone.
- ✓ Centralized networking.
- ✓ Shared security services.
- ✓ Consistent guardrails & compliance.

ENTERPRISE LANDING ZONE**5. BEST PRACTICES**

- ✓ Use least-privilege IAM roles.
- ✓ Use External ID for cross-account access.
- ✓ Centralize logs in a dedicated account.
- ✓ Enforce guardrails with SCPs.
- ✓ Tag everything consistently.
- ✓ Automate account provisioning.
- ✓ Monitor & audit continuously.

CROSS-ACCOUNT ACCESS FLOW**EXAMPLE: ASSUME ROLE (CLI)**

```

> aws sts assume-role \
  --role-arn arn:aws:iam::444444444444:role/crossaccount-role \
  --role-session-name veriqta-session \
  --external-id veriqta-external-id

Export credentials to use with Terraform/AWS CLI.
  
```

COMMON MISTAKES

- ✗ Hardcoding account IDs.
- ✗ Using long-lived access keys.
- ✗ Not using External ID.
- ✗ Over-permissive roles and policies.
- ✗ No centralized logging or monitoring.

**VALIDATION CHECKLIST**

- ✓ SCPs applied & tested.
- ✓ Cross-account roles configured.
- ✓ Provider aliases working.
- ✓ Resources deployed to correct accounts.
- ✓ Logging & monitoring in place.

**PRO TIP**

Build reusable modules with provider aliases and remote state for scalable, enterprise-grade deployments.



VERIQTA

13. MULTI-REGION DEPLOYMENTS

REMEMBER!

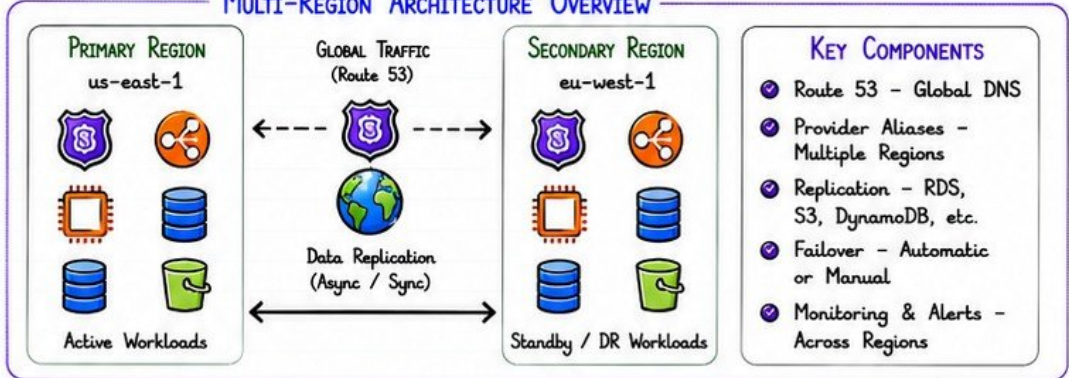
- ✓ Design for failure.
- ✓ Deploy for resilience.
- ✓ Test your failover.
- ✓ Automate recovery.



★ WHY MULTI-REGION?

- ✓ High availability & reliability
- ✓ Protects against region failures
- ✓ Lower latency for global users
- ✓ Meets compliance & data residency requirements
- ✓ Stronger disaster recovery

MULTI-REGION ARCHITECTURE OVERVIEW



1. HIGH AVAILABILITY

- ✓ Deploy critical workloads across multiple regions.
- ✓ Remove single points of failure.
- ✓ Use Health Checks to detect issues.
- ✓ Keep services stateless where possible.



2. PROVIDER ALIASES

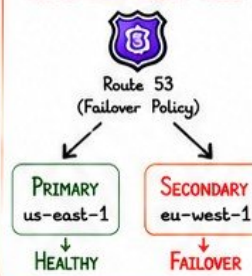
- ✓ Use provider aliases to manage multiple regions.
- ✓ Each alias maps to a specific region.
- ✓ Keeps resources isolated per region.

```
provider "aws" {  
  region = "us-east-1"  
  alias = "use1"  
}  
  
provider "aws" {  
  region = "eu-west-1"  
  alias = "euw1"  
}  
  
# Usage  
resource "aws_s3_bucket" "primary" {  
  provider = aws.use1  
  bucket = "app-primary-bucket"  
}  
  
resource "aws_s3_bucket" "dr" {  
  provider = aws.euw1  
  bucket = "app-dr-bucket"  
}
```

3. FAILOVER STRATEGY

- ✓ Automatic Failover: Fast recovery using Route 53 health checks.
- ✓ Manual Failover: Switch DNS to secondary region.
- ✓ Use weighted or failover routing policies.
- ✓ Test failover regularly!

ROUTE 53 FAILOVER POLICY



4. DISASTER RECOVERY

- ✓ Replicate data to secondary region.
- ✓ Use backup & snapshots across regions.
- ✓ Automate infrastructure with Terraform.
- ✓ Document recovery runbooks.
- ✓ Perform DR drills regularly.



5. BEST PRACTICES

- ✓ Design for failure from day one.
- ✓ Keep regions in different zones.
- ✓ Automate failover & recovery.
- ✓ Monitor health across regions.
- ✓ Optimize cost with right replication strategy.



EXAMPLE: TWO-REGION DEPLOYMENT FLOW



COMMON MISTAKES

- ✗ Deploying everything in one region.
- ✗ Not testing failover before failure happens.
- ✗ Forgetting to replicate data.
- ✗ Hardcoding region values.
- ✗ Ignoring cross-region costs.



DR CHECKLIST



- ✓ Multi-region infrastructure defined in Terraform
- ✓ Data replication configured & tested
- ✓ Route 53 failover or latency policy
- ✓ Backups & snapshots across regions
- ✓ Failover tested & documented
- ✓ Monitoring & alerts across regions
- ✓ DR runbook created and reviewed

PRO TIP

Resilience is not a feature, it's an architecture choice. Design, test, and automate your recovery before you need it.



VERIQTA

14. TERRAFORM PERFORMANCE

REMEMBER!

- ✓ Optimize for speed.
- ✓ Optimize for reliability.
- ✓ Keep state efficient.
- ✓ Automate best practices.

★ WHY PERFORMANCE MATTERS?

- ✓ Large infrastructure = slow operations.
- ✓ Slow plans increase risk of errors.
- ✓ Faster pipelines improve productivity.
- ✓ Lower state size improves reliability.
- ✓ Better performance reduces costs.

FACTORS THAT AFFECT TERRAFORM PERFORMANCE

STATE SIZE



Large state files slow down plan, apply and refresh.

PROVIDERS



Provider downloads and schema loads add time.

MODULES



Complex or inefficient modules increase runtime.

PARALLELISM



Too low = slow
Too high = API throttling/errors.

DATA SOURCES



Many data sources slow down plan and refresh.

NETWORK



Latency impacts provider calls and backends.

1. LARGE STATE OPTIMIZATION

- ✓ Split state by environment, region or team.
- ✓ Use workspaces only for isolated use cases.
- ✓ Avoid putting everything in one state.
- ✓ Use remote state for data sharing.

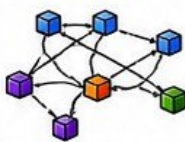
EXAMPLE STRUCTURE

```
infrastructure/  
├── envs/  
│   ├── prod/  
│   │   ├── vpc.tfstate  
│   │   └── eks.tfstate  
│   └── staging/  
│       ├── vpc.tfstate  
│       └── eks.tfstate  
└── modules/  
    ├── vpc/  
    └── eks/
```

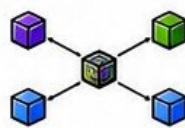
2. MODULE OPTIMIZATION

- ✓ Keep modules small and focused.
- ✓ Avoid deep nesting.
- ✓ Use count/for_each efficiently.
- ✓ Pass only necessary variables.
- ✓ Use outputs wisely.

BAD (AVOID)



GOOD (PREFERRED)



3. PARALLELISM TUNING

- ✓ Increase parallelism for faster applies.
- ✓ Avoid too high values (API limits).
- ✓ Default: 10
- ✓ Recommended: 20 - 50
- ✓ Test and find the sweet spot.

SET PARALLELISM

```
# CLI  
terraform apply  
-parallelism=30  
  
# In tfc/terraform.rc  
parallelism = 30
```



4. PROVIDER CACHING

- ✓ Enable provider plugin caching.
- ✓ Reduces download time in CI/CD.
- ✓ Use local cache directory.
- ✓ Speeds up init and plan.

PROVIDER CACHE

```
# CLI  
terraform init  
-plugin-dir=.terraform.d  
-reconfigure
```

CLI CONFIG

```
# ~/.terraformrc  
plugin_cache_dir =  
"$HOME/.terraform.d/  
plugin-cache"
```

5. REDUCE DATA SOURCE OVERHEAD

- ✓ Avoid unnecessary data sources.
- ✓ Use locals and variables when possible.
- ✓ Cache results with locals.
- ✓ Minimize API calls.

EXAMPLE (GOOD)

```
locals {  
  region = var.region  
  account_id = data.aws_caller_identity.current.account_id  
}
```

AVOID (EXCESSIVE)

```
data "aws_ami" "latest" {  
  most_recent = true  
  # called multiple times  
  # across many modules  
}
```

6. BEST PRACTICES

- ✓ Keep state small and modular.
- ✓ Use remote state backend.
- ✓ Enable locking.
- ✓ Use version pinning for providers.
- ✓ Run terraform in a stable network.
- ✓ Monitor and review performance regularly.



PERFORMANCE BOOST TIPS



Keep state files under 100MB per state.



Break infrastructure into small, reusable modules.



Increase parallelism (20-50).



Use provider caching for faster init.



Minimize data sources and external calls.



Use remote backend with locking (S3 + DynamoDB).



Monitor execution time and optimize continuously.

COMMON MISTAKES

- ✗ One huge state for everything.
- ✗ Too many data sources.
- ✗ Low parallelism settings.
- ✗ No provider caching in CI/CD.
- ✗ Ignoring provider version pinning.



CHECKLIST



- ✓ State split and optimized.
- ✓ Modules small and reusable.
- ✓ Parallelism tuned.
- ✓ Provider caching enabled.
- ✓ Remote backend with locking.
- ✓ Performance reviewed regularly.

PRO TIP

Performance is an ongoing practice. Test, measure, and optimize. Small improvements today save hours in every pipeline!



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

VERIQTA

15. TERRAGRUNT

REMEMBER!

- ✓ Keep configurations DRY.
- ✓ Manage complexity at scale.
- ✓ Promote consistency.
- ✓ Speed up deployments.

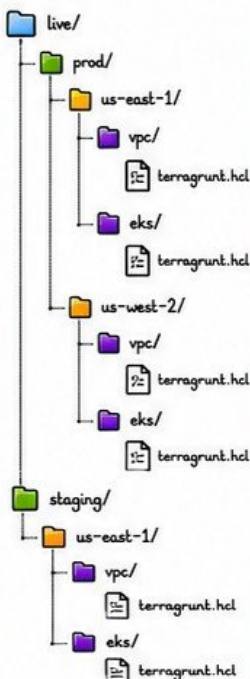
★ WHAT IS TERRAGRUNT?

- ✓ Terragrunt is a thin wrapper for Terraform that provides extra tools for working with multiple Terraform modules, remote state, and complex infrastructure.
- ✓ It helps you keep your configurations DRY, organized and scalable.

WHY TERRAGRUNT?



1. FOLDER HIERARCHY



BENEFITS

- ✓ Easy navigation
- ✓ Environment isolation
- ✓ Scales with your org

2. DRY CONFIGURATION

Use root terragrunt.hcl to define common settings for all environments.

```
Root: terragrunt.hcl

remote_state {
  backend = "s3"
  config = {
    bucket = "veriqta-terraform"
    key    = "${path_relative_to_include()}/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
  }
}

inputs = {
  project_name = "veriqta"
  owner        = "platform-team"
  environment  = "prod"
  tags = {
    managed_by = "terragrunt"
    project    = "veriqta"
  }
}
```

```
Child terragrunt.hcl

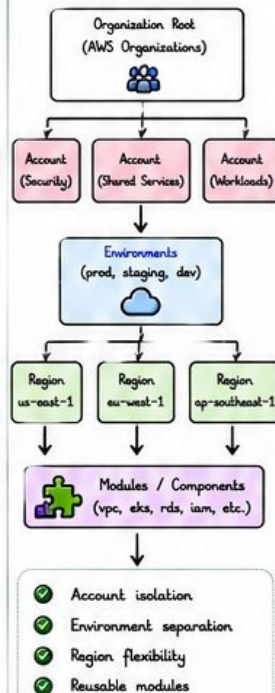
include "root" {
  path = find_in_parent_folders()
}

terraform {
  source = "git::https://.../vpc.git"
}

inputs = {
  vpc_cidr = "10.0.0.0/16"
}
```

3. ENTERPRISE STRUCTURE

Organize by account and environment for visibility and control.



4. HOW TERRAGRUNT WORKS



KEY ADVANTAGE

Terragrunt handles dependencies and remote state automatically!

5. BEST PRACTICES

- ✓ Use remote state for all environments.
- ✓ Keep root terragrunt.hcl minimal and clean.
- ✓ Use version pinning for Terraform and providers.
- ✓ Structure folders by account, region and environment.
- ✓ Use inputs and locals wisely.
- ✓ Avoid hardcoding. Use variables and secrets.
- ✓ Run terragrunt run-all for full stacks.
- ✓ Review plans before applying.
- ✓ Keep modules small and reusable.

TERRAGRUNT COMMANDS CHEAT SHEET



```
terragrunt init
Initialize a module

terragrunt validate
Validate configuration

terragrunt plan
Run plan for current stack
```

```
terragrunt apply
Apply changes

terragrunt run-all plan
Run plan for all modules

terragrunt run-all apply
Apply for all modules
```

```
terragrunt run-all destroy
Destroy all resources

terragrunt graph-dependencies
Visualize dependencies

terragrunt hclfmt
Format terragrunt.hcl files
```

PRO TIP

Start small, define a solid structure, and let Terragrunt handle the complexity!

COMMON MISTAKES

- ✗ Not using remote state.
- ✗ Hardcoding values.
- ✗ Ignoring dependency order.
- ✗ Too many variables in root.
- ✗ Not reviewing plans.



CHECKLIST



- ✓ Root configuration defined.
- ✓ Remote state configured.
- ✓ Folder structure in place.
- ✓ Modules reusable and tested.
- ✓ Plans reviewed before apply.
- ✓ Version pinning in place.

FINAL NOTE

Terragrunt brings order, consistency, and scalability to your Terraform deployments. Use it to build enterprise-grade infrastructure with confidence!



VERIQTA

16. PRODUCTION DEPLOYMENT STRATEGY

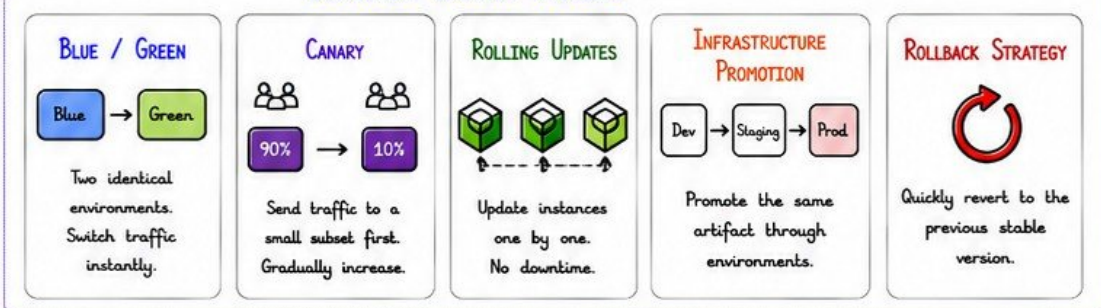
REMEMBER!

- ✓ Deploy safely.
- ✓ Minimize risk.
- ✓ Enable fast rollback.
- ✓ Automate & monitor.

★ WHY DEPLOYMENT STRATEGY?

- ✓ Reduce downtime & risk.
- ✓ Detect issues early.
- ✓ Deliver value continuously.
- ✓ Enable quick rollback.
- ✓ Improve reliability & customer experience.

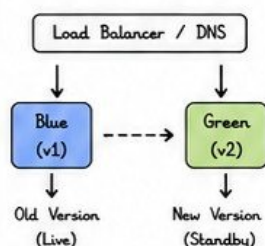
DEPLOYMENT STRATEGY OVERVIEW



1. BLUE / GREEN DEPLOYMENT

Steps:

- 1 Deploy new version to Green.
- 2 Run tests and validations.
- 3 Switch traffic from Blue to Green.
- 4 Keep Blue as fallback.
- 5 Decommission old Blue.



PROS:

- ✓ Zero downtime.
- ✓ Easy rollback.
- ✓ Full validation before cutover.

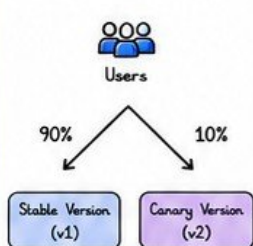
CONS:

- ✗ Double infrastructure cost.
- ✗ Longer deployment time.

2. CANARY DEPLOYMENT

Steps:

- 1 Deploy new version.
- 2 Route a small % of traffic (e.g., 5% to 10%).
- 3 Monitor metrics & logs.
- 4 Gradually increase traffic.
- 5 Rollback if issues detected.



PROS:

- ✓ Detect issues early.
- ✓ Lower risk.
- ✓ Real user feedback.

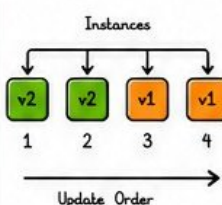
CONS:

- ✗ Complex setup.
- ✗ Requires good monitoring.

3. ROLLING UPDATES

Steps:

- 1 Update instance #1.
- 2 Wait for health check.
- 3 Update instance #2.
- 4 Continue until all instances updated.



PROS:

- ✓ No downtime.
- ✓ Simple & native.
- ✓ Works for most apps.

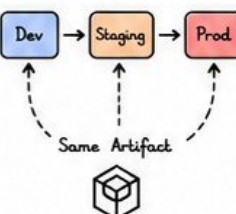
CONS:

- ✗ Slower rollout.
- ✗ Issues may appear later.

4. INFRASTRUCTURE PROMOTION

Steps:

- 1 Build artifact once.
- 2 Deploy to Dev.
- 3 Test & validate.
- 4 Promote to Staging.
- 5 Validate again.
- 6 Promote to Prod.



PROS:

- ✓ Consistency across envs.
- ✓ Reproducible deployments.
- ✓ Audit & compliance friendly.

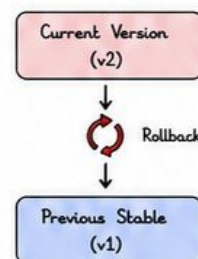
CONS:

- ✗ Requires pipeline setup.
- ✗ Needs strict version control.

5. ROLLBACK STRATEGY

Steps:

- 1 Detect issue (alerts/metrics).
- 2 Trigger rollback.
- 3 Restore previous version.
- 4 Validate system.
- 5 Document incident.



PROS:

- ✓ Minimize impact.
- ✓ Restore stability fast.
- ✓ Builds user trust.

CONS:

- ✗ May lose recent changes.
- ✗ Rollback must be tested.

DEPLOYMENT STRATEGY COMPARISON

STRATEGY	DOWNTIME	RISK LEVEL	COMPLEXITY	BEST USE CASE
Blue / Green	None	Low	Medium	Major releases, critical systems
Canary	None	Very Low	High	Web apps, microservices
Rolling Updates	None	Low	Low	Stateless apps, containers
Infra Promotion	None	Low	High	Enterprise, regulated environments
Rollback	N/A	N/A	Low	All deployments (safety net)

★ BEST PRACTICES

- ✓ Automate deployments with CI/CD.
- ✓ Use Infrastructure as Code.
- ✓ Test in lower environments first.
- ✓ Monitor every deployment.
- ✓ Keep rollback plan ready.
- ✓ Document every change.

COMMON MISTAKES

- ✗ Skipping tests before production.
- ✗ No rollback plan.
- ✗ Ignoring monitoring & alerts.
- ✗ Deploying too fast.

CHECKLIST



- ✓ Code tested & reviewed.
- ✓ Infra validated.
- ✓ Monitoring in place.
- ✓ Rollback plan ready.
- ✓ Stakeholders notified.

PRO TIP

Start small, automate everything, and always plan your rollback before you deploy!



VERIQTA

17. ENTERPRISE MODULE DESIGN

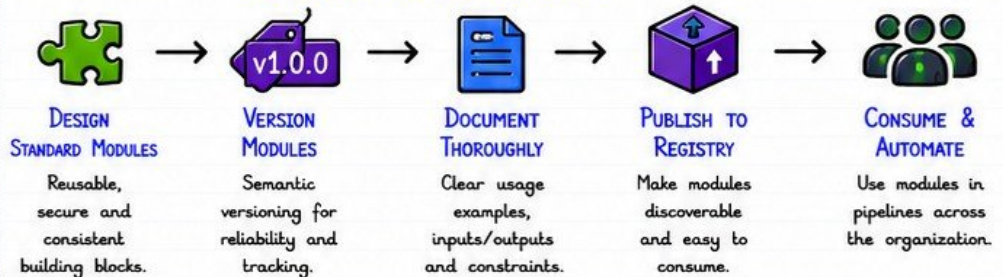
REMEMBER!

- ✓ Build once, use everywhere.
- ✓ Keep modules consistent.
- ✓ Document everything.
- ✓ Version and publish.
- ✓ Automate and reuse.

★ WHY ENTERPRISE MODULES?

- ✓ Ensure consistency across teams
- ✓ Reduce duplication and errors
- ✓ Speed up deployments
- ✓ Enforce best practices
- ✓ Simplify maintenance
- ✓ Scale across the organization

ENTERPRISE MODULE LIFECYCLE



1. MODULE STANDARDS

- ✓ Consistent naming conventions
- ✓ Standard input/output patterns
- ✓ Security by default
- ✓ Tagging and labeling
- ✓ Least privilege access

EXAMPLE STRUCTURE

```
modules/  
├── vpc/  
│   ├── main.tf  
│   ├── variables.tf  
│   ├── outputs.tf  
│   ├── versions.tf  
│   ├── README.md  
│   └── examples/  
│       └── basic/  
│           └── main.tf
```

GOOD PRACTICE

- ✓ Keep modules small, focused and composable.

AVOID

- ✗ Monolithic modules that are hard to maintain and reuse.

2. VERSIONING

Use Semantic Versioning (SemVer).

SEMVER GUIDE

MAJOR (X.y.z)

Breaking changes

MINOR (x.Y.z)

New features, backward compatible

PATCH (x.y.Z)

Bug fixes, backward compatible

VERSION EXAMPLE

```
module "vpc" {  
  source = "git::ssh://  
git@gitlab.com/veriqta/  
terraform-aws-vpc.git"  
  version = "~> 2.1.0"  
}
```

BEST PRACTICE

- ✓ Pin versions.
- ✓ Avoid using "latest".
- ✓ Update versions in stages.

3. DOCUMENTATION

Every module must be well documented.

- ✓ README with purpose and examples
- ✓ Input variable table
- ✓ Output value table
- ✓ Dependencies
- ✓ Assumptions and limitations

README MUST INCLUDE

- 🎯 Purpose
- 📄 Usage
- ➡ Inputs
- ➡ Outputs
- 📄 Examples
- ⚙ Requirements
- ❓ FAQs

EXAMPLE USAGE

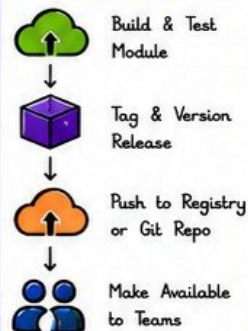
```
module "s3_bucket" {  
  source = "git::ssh://  
git@gitlab.com/veriqta/  
terraform-aws-s3.git"  
  bucket_name = "my-bucket"  
  version = "~> 1.0.0"  
}
```

4. PUBLISH MODULES

Publish modules for easy discovery and reuse.

- ✓ Use internal registry or VCS
- ✓ Provide clear versions and changelog
- ✓ Automate release process
- ✓ Ensure access control

PUBLISH FLOW



5. INTERNAL REGISTRIES

Store and share modules securely within your organization.

- ✓ Private Git repositories (GitHub, GitLab, Bitbucket)
- ✓ Terraform Registry (Private)
- ✓ Access control (RBAC)
- ✓ Audit and monitoring
- ✓ High availability

REGISTRY OPTIONS

- GitHub Enterprise
- GitLab Self-Managed
- Terraform Registry (Private)
- AWS CodeArtifact
- Azure Artifacts

GOOD PRACTICE

- ✓ Centralize modules for consistency, security and governance.

ENTERPRISE MODULE DESIGN PRINCIPLES



SECURITY BY DEFAULT

Secure inputs, enforce least privilege.



CONSISTENCY

Follow standards across all modules.



REUSABILITY

Design modules to be generic and reusable.



SCALE

Build modules that scale with the organization.



TESTING

Automate tests for reliability and quality.



GOVERNANCE

Version, audit and control access.



TEAM ALIGNMENT

Align with team needs and feedback.

COMMON MISTAKES

- ✗ Hardcoding values in modules.
- ✗ Overly complex modules.
- ✗ Missing documentation.
- ✗ Not versioning modules.
- ✗ Breaking changes without notice.



CHECKLIST

- ✓ Module standards followed.
- ✓ All inputs/outputs documented.
- ✓ Examples provided and tested.
- ✓ Version pinned and released.
- ✓ Published to internal registry.
- ✓ Access controls configured.
- ✓ Teams informed and enabled.

PRO TIP

Great modules are the foundation of scalable infrastructure and high-performing teams.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

VERIQTA

18. TERRAFORM SECURITY REVIEW

REMEMBER!

- ✓ Security is everyone's job.
- ✓ Principle of least privilege.
- ✓ Protect state and secrets.
- ✓ Review code and dependencies.
- ✓ Automate checks and monitoring.

★ WHY SECURITY MATTERS?

- ✓ Terraform manages critical infrastructure.
- ✓ Misconfigurations can expose data.
- ✓ State files may contain sensitive info.
- ✓ Compromised credentials = full account access.
- ✓ Secure code = secure infrastructure.

TERRAFORM SECURITY OVERVIEW



LEAST PRIVILEGE

Use minimal permissions required to perform the task.



IAM DESIGN

Design secure roles, policies and assume roles safely.



STATE SECURITY

Protect state files, encrypt and control access strictly.



CODE REVIEW

Review code for security issues and best practices.



SECURITY CHECKLIST

Follow a consistent checklist before every deployment.

1. LEAST PRIVILEGE

- ✓ Grant only required actions.
- ✓ Avoid AdministratorAccess.
- ✓ Use resource-level permissions.
- ✓ Separate roles per environment.
- ✓ Use conditions (tags, MFA, source IP).

EXAMPLE POLICY

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3::my-bucket",
        "arn:aws:s3::my-bucket/*"
      ]
    }
  ]
}
```

AVOID

- ✗ * on * permissions
- ✗ Over-privileged roles
- ✗ Using root credentials

2. IAM DESIGN

- ✓ Use dedicated IAM roles for Terraform.
- ✓ Attach minimal policies.
- ✓ Use AssumeRole for cross-account.
- ✓ Enable MFA for humans.
- ✓ Rotate credentials regularly.

BEST PRACTICES

- One role per use case
- Use groups for policy management
- Enforce MFA
- No long-lived access keys
- Use cross-account roles instead of keys

3. STATE SECURITY

- ✓ Store state in encrypted remote backend.
- ✓ Enable versioning.
- ✓ Restrict bucket access.
- ✓ Enable DynamoDB state locking.
- ✓ Encrypt in transit (TLS) and at rest.

SECURE BACKEND EXAMPLE

```
terraform {
  backend "s3" {
    bucket = "tf-state-prod"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    dynamodb_table = "tf-locks"
  }
}
```

CHECK LIST

- ✓ S3 bucket encrypted (SSE-KMS)
- ✓ Bucket versioning enabled
- ✓ Bucket access logging enabled
- ✓ DynamoDB locking enabled
- ✓ Bucket policy restricts access
- ✓ No public access

4. CODE REVIEW

- ✓ Review all Terraform changes.
- ✓ Look for security misconfigurations.
- ✓ Validate input variables.
- ✓ Check provider version and pin.
- ✓ Scan for hardcoded secrets.

WHAT TO REVIEW

- ✓ IAM policies and roles
- ✓ Security Groups / NACLs
- ✓ S3 bucket policies
- ✓ KMS key policies
- ✓ Public access settings
- ✓ Secrets in code
- ✓ Resource tagging
- ✓ Network exposure
- ✓ Outputs with sensitive data

5. SECURITY CHECKLIST

- ✓ Using least privilege IAM?
- ✓ State stored securely?
- ✓ State locking enabled?
- ✓ S3 bucket private?
- ✓ Encryption enabled?
- ✓ Versioning enabled?
- ✓ MFA enabled for users?
- ✓ No hardcoded secrets?
- ✓ Provider versions pinned?
- ✓ Terraform version pinned?
- ✓ Code reviewed?
- ✓ Plans reviewed?
- ✓ CI/CD security checks?
- ✓ Monitoring & alerts enabled?
- ✓ Audit logging enabled?
- ✓ Access reviewed regularly?

SECURE TERRAFORM WORKFLOW



Write Code



Review Code



Run Security Scans
(tfsec, checkov)



Plan & Review



Apply with Least
Privilege Role



Monitor &
Audit



Continuously
Improve

COMMON MISTAKES

- ✗ Using AdministratorAccess
- ✗ Storing state locally
- ✗ Exposing state bucket
- ✗ Hardcoding secrets
- ✗ Skipping code review



SECURITY TOOLS

- ✓ tfsec
- ✓ Checkov
- ✓ Terrascan
- ✓ Snyk IaC
- ✓ AWS Config
- ✓ CloudTrail

CHECKLIST SUMMARY

- ✓ Access is least privilege
- ✓ State is encrypted & locked
- ✓ Code is reviewed
- ✓ No secrets in code
- ✓ Dependencies are up-to-date
- ✓ Monitoring & audit enabled



PRO TIP

Security is not a one-time activity. Automate, monitor and keep improving!



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

VERIQTA

19. REAL PRODUCTION CASE STUDIES

REMEMBER!

- ✓ Real incidents teach the most.
- ✓ Understand impact & root cause.
- ✓ Learn what to do differently.
- ✓ Share lessons with your team.
- ✓ Document and improve.

★ WHY CASE STUDIES?

- ✓ Learn from real incidents.
- ✓ Understand blast radius.
- ✓ Improve troubleshooting skills.
- ✓ Prevent similar mistakes.
- ✓ Build a culture of learning.

PRODUCTION INCIDENT CATEGORIES



STATE CORRUPTION

State file becomes invalid or corrupted, causing failures.



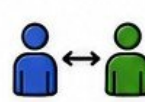
FAILED DEPLOYMENT

Changes break environments and cause outages.



DRIFT INCIDENT

Manual changes create drift leading to unexpected issues.



CROSS-ACCOUNT FAILURE

Permissions or trust issues break cross-account operations.



LESSONS LEARNED

Document key takeaways and actions to prevent recurrence.

1. STATE CORRUPTION



WHAT HAPPENED

The Terraform state file was manually edited and became corrupted.

IMPACT

- ✗ Plan failed for all environments
- ✗ Deployments were blocked
- ✗ Team productivity impacted

ROOT CAUSE

Manual changes to the state file outside Terraform.

HOW IT WAS FIXED

- ✓ Restored from S3 backup
- ✓ Ran 'terraform state pull'
- ✓ Validated and re-applied

LESSONS LEARNED

- 💡 Never edit state manually
- 💡 Enable versioning
- 💡 Use state locking
- 💡 Automate backups

2. FAILED DEPLOYMENT



WHAT HAPPENED

A new release removed a required security group rule, breaking application connectivity.

IMPACT

- ✗ Users could not access app
- ✗ Increased error rates
- ✗ Customer complaints

ROOT CAUSE

Insufficient testing and missing validation in pipeline.

HOW IT WAS FIXED

- ✓ Rolled back to last stable version
- ✓ Added automated tests
- ✓ Updated pipeline checks

LESSONS LEARNED

- 💡 Test in lower environments
- 💡 Use validation & policy checks
- 💡 Implement blue/green or canary deployments

3. DRIFT INCIDENT



WHAT HAPPENED

An engineer manually changed an RDS parameter in production.

IMPACT

- ✗ Configuration drift detected
- ✗ Unexpected behavior
- ✗ Next Terraform apply failed

ROOT CAUSE

Lack of change control and manual modifications.

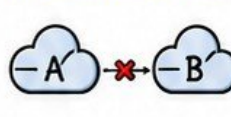
HOW IT WAS FIXED

- ✓ Reverted manual change
- ✓ Re-imported resources
- ✓ Enabled drift detection

LESSONS LEARNED

- 💡 Enforce change management
- 💡 Enable drift detection
- 💡 Use IaC for all changes

4. CROSS-ACCOUNT FAILURE



WHAT HAPPENED

Terraform in Account A could not assume role in Account B due to expired trust policy.

IMPACT

- ✗ Resources not created
- ✗ Pipeline failed
- ✗ Delayed release

ROOT CAUSE

Expired IAM role trust relationship.

HOW IT WAS FIXED

- ✓ Updated trust policy
- ✓ Tested assume-role
- ✓ Re-ran pipeline successfully

LESSONS LEARNED

- 💡 Monitor IAM roles
- 💡 Automate role validation
- 💡 Document expiration dates

5. LESSONS LEARNED



WHAT HAPPENED

Post-incident review revealed multiple process and tooling gaps.

IMPACT

- ✗ Repeat issues risk
- ✗ Knowledge gaps
- ✗ No standard process

ROOT CAUSE

Lack of documentation and inconsistent practices.

HOW IT WAS FIXED

- ✓ Created runbooks
- ✓ Standardized workflows
- ✓ Trained the team

LESSONS LEARNED

- 💡 Document everything
- 💡 Share knowledge
- 💡 Continuously improve

KEY TAKEAWAYS



Prevent issues with standards and automation.



Detect problems early with monitoring.



Backup and test recovery plans regularly.



Secure access with least privilege.



Share knowledge and learn as a team.

REAL IMPACT NUMBERS (EXAMPLE)

- ✓ State corruption downtime: 8+ hours
- ✓ Failed deployment outage: 3 hours
- ✓ Drift incident resolution: 5 hours
- ✓ Cross-account failure delay: 2 days
- ✓ Customer impact: 1000+ users affected



COMMON MISTAKES

- ✗ Editing state manually.
- ✗ Skipping code review.
- ✗ Not testing in lower environments.
- ✗ Ignoring drift.
- ✗ Overly permissive IAM roles.



INCIDENT RESPONSE FLOW



1. Detect
Alert triggers or user report



2. Investigate
Check logs, metrics, events



3. Fix
Implement corrective action



4. Validate
Confirm system is healthy



5. Review
Document and share lessons

PRO TIP

The best engineers learn from every incident and make the next one stronger.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

VERIQTA

20. VOLUME 2

CAPSTONE PROJECT

REMEMBER!

- Think like an architect.
- Automate everything.
- Secure by design.
- Validate continuously.
- Document your work.

CAPSTONE GOAL

Build a production-ready, secure, scalable, multi-region AWS platform using best practices and Terraform ecosystem tools.

BUILD A COMPLETE ENTERPRISE AWS PLATFORM



1. MULTI-ACCOUNT ARCHITECTURE

Secure account structure using AWS Organizations, SCPs and IAM best practices.



2. REUSABLE MODULES

Build reusable, versioned modules for VPC, EKS, RDS, ALB, IAM, etc.



3. REMOTE STATE

Store state in S3 with DynamoDB locking, encryption and versioning.



4. TERRAGRUNT

Manage environments (DRY) with Terragrunt, remote state, and hierarchical config.



5. CI/CD PIPELINE

Automate plan, validate, and apply using GitHub Actions or GitLab CI.



6. SECRETS MANAGER

Store and rotate secrets securely using AWS Secrets Manager and IAM roles.



7. POLICY AS CODE

Enforce policies with Sentinel/OPA, tfsec, Checkov and Conftest.



8. MULTI-REGION DEPLOYMENT

Deploy across multiple regions for high availability and disaster recovery.



9. VALIDATION CHECKLIST

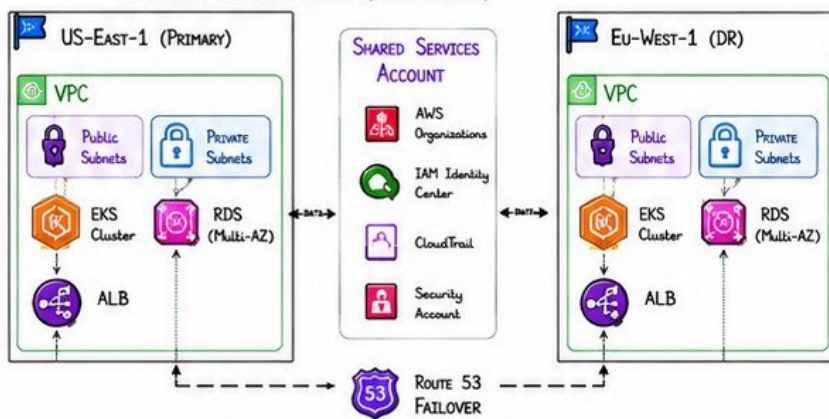
Automated validations to ensure security, compliance and reliability.



10. SENIOR ENGINEER INTERVIEW QUESTIONS

Test your knowledge with real-world scenario based questions.

SOLUTION ARCHITECTURE (HIGH LEVEL)



CAPSTONE VALIDATION CHECKLIST

- All accounts created and governed with SCPs
- Network connectivity validated (VPC peering / TGW)
- Remote state stored in S3 with DynamoDB lock
- All modules are reusable and documented
- CI/CD pipeline runs plan, validate and apply
- Secrets stored in Secrets Manager (no hardcoded secrets)
- Policies enforced with Policy as Code
- Resources deployed in multiple regions
- Monitoring, logging and alerts configured
- Cost tags and budgets configured
- Security baseline and compliance validated

SENIOR ENGINEER INTERVIEW QUESTIONS

- How would you design a multi-account AWS strategy for a large enterprise?
- How do you handle Terraform state in a secure and scalable way?
- How would you enforce security and compliance in Terraform?
- How would you manage secrets in a Terraform environment?
- How do you handle changes across multiple environments?
- How would you recover from a corrupted Terraform state?
- How do you ensure high availability for infrastructure?
- How do you control costs and enforce tagging?

DELIVERABLES

- Multi-account AWS platform
- Reusable Terraform modules
- Terragrunt environment structure
- CI/CD pipeline (GitHub/GitLab)
- Secure remote state setup
- Policy as Code implementation
- Multi-region deployment
- Documentation & runbooks

SUCCESS PRINCIPLES

- Automate and standardize everything.
- Security is built-in, not added later.
- Reusable modules save time.
- Validate early and continuously.
- Document for your future self.

"The best engineers build systems that scale, heal and last."



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA